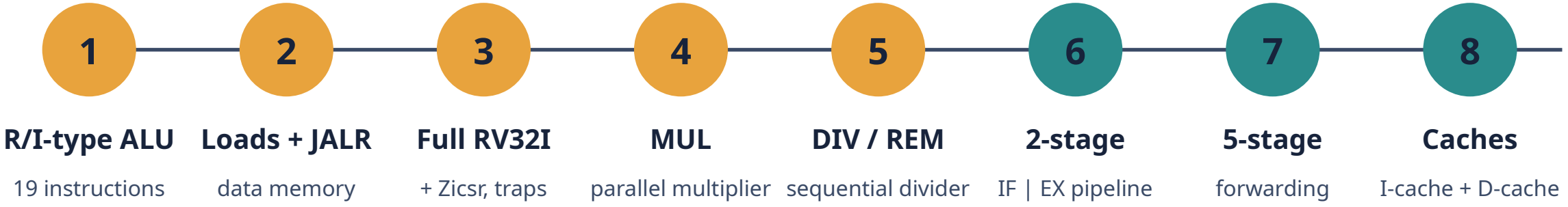
Building a RISC-V core in Chisel, stage by stage

From 19 RV32I ALU instructions to a 5-stage RV32IM pipeline with I- and D-caches

1 2 3 4 5 6 7 8

SmartComputerLab

The journey: eight stages, one idea each



Stages 1–5 · grow the ISA

Single-cycle datapath: every instruction in one clock, except a division (34 cycles).

Stages 6–8 · speed up the same ISA

RV32IM + Zicsr unchanged; the reference model and all program tests carry over.

SmartComputerLab

Three rules kept every stage in a known-good state

1

One idea per stage

Each project adds exactly one mechanism to a design that already passes all its tests, so a failure points at the new hardware.

2

A fixed reference model

RefModel executes the ISA instruction by instruction in Scala. The core runs next to it and every retirement is compared.

3

A stable trace contract

One trace record per cycle: pc, next pc, instruction, register written, memory access, trap and cause, in every stage.

One toolchain and one project kit for all eight stages

Chisel 3.6.1

Hardware in Scala 2.13, with the chisel3 compiler plugin

chiseltest 0.6.2

Simulation on Treadle: pure Scala, nothing to install

sbt 1.10.2

Build, test and run on Java 11 or newer

GTKWave

VCD waveforms with ready-made core.gtkw sessions

CIRCT firtool

Optional: SystemVerilog through firtool 1.37.0

Icarus / Verilator

Run the Verilog testbenches of the split modules

Each project contains

- One Scala file per hardware block
- A core and a test wrapper with memories from program.hex / data.hex
- Unit, lockstep, timing and waveform tests
- Emit: Verilog generation
- README and DESIGN_NOTES

SmartComputerLab

1–5

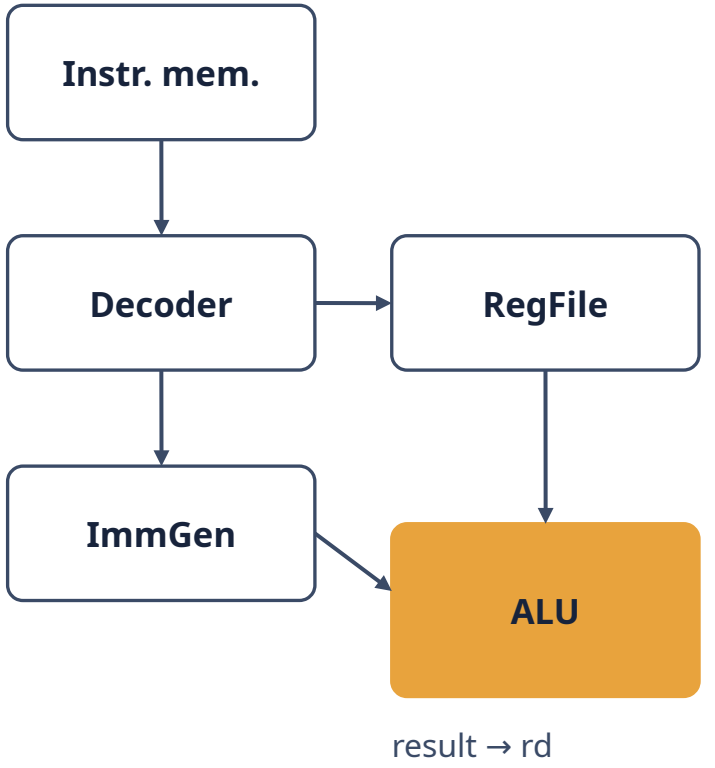
Stages 1–5: single-cycle cores

Growing the instruction set from 19 ALU instructions to RV32IM + Zicsr

RV32I ALU core: 19 instructions in one cycle

1 **Stage 1**

- 10 R-type: ADD SUB SLL SLT SLTU XOR SRL SRA OR AND
- 9 I-type: ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI
- ALU (10 operations), decoder, sign-extended I immediate, 32 × 32 register file with x0 = 0
- pc + 4 every cycle; an unknown word runs as a no-op with valid = 0

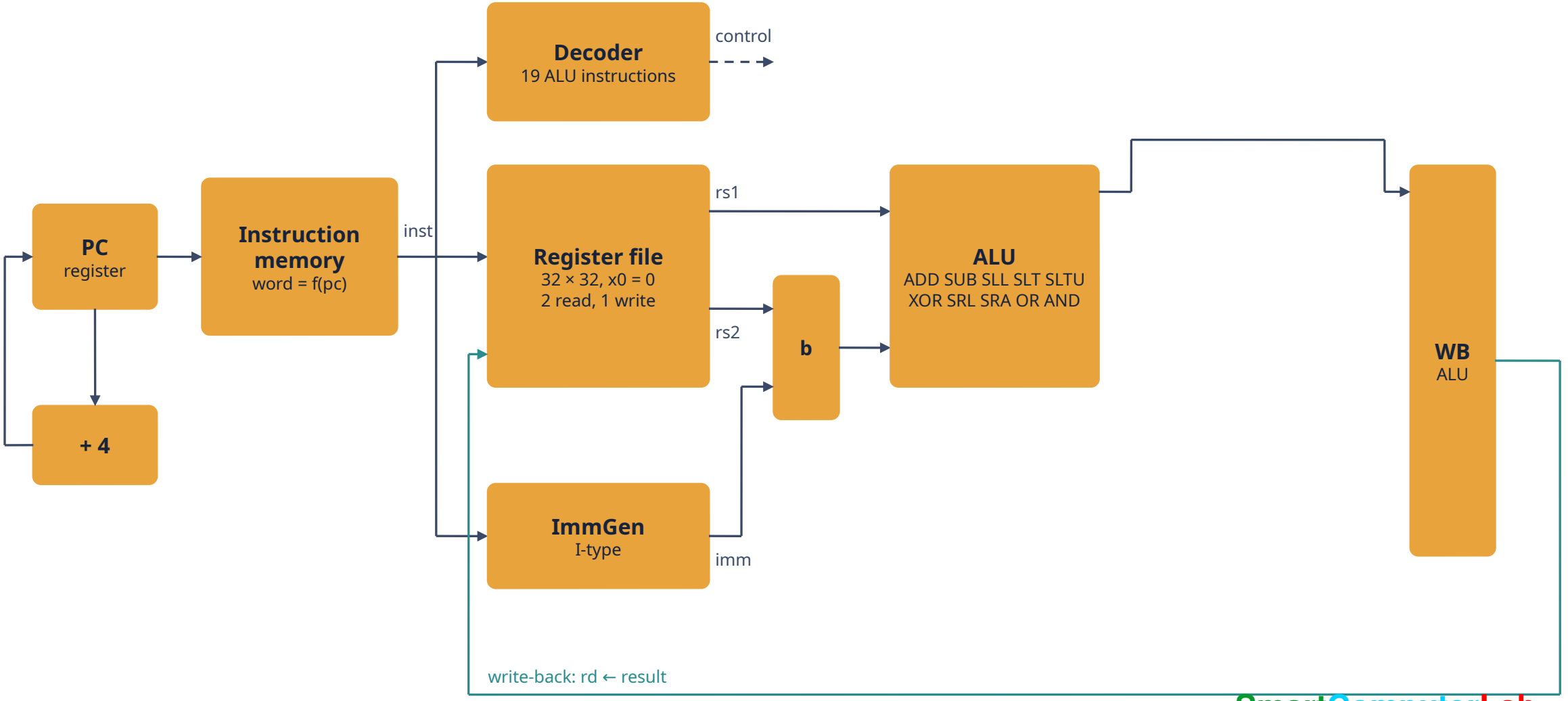


19
instructions

1
cycle each

32
tests

Stage 1 architecture: a single-cycle ALU datapath



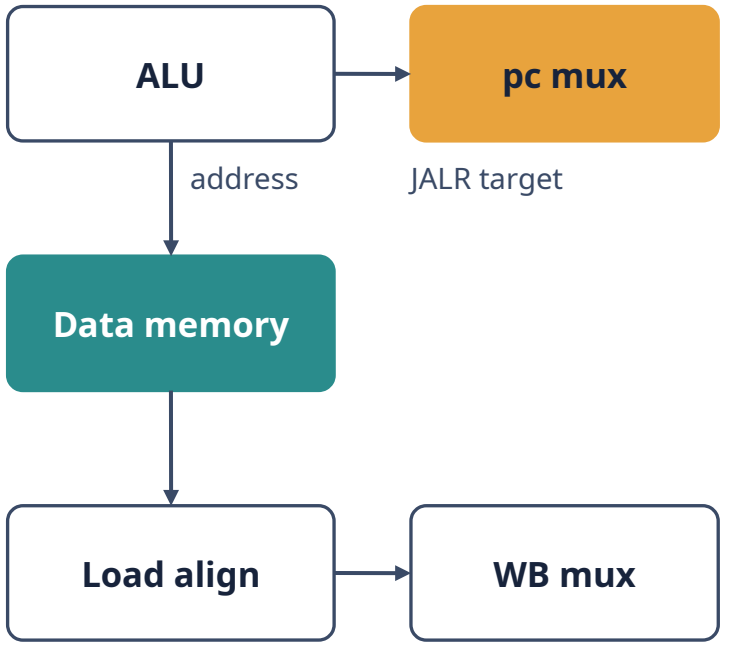
new in this stage carried over write-back

SmartComputerLab

Loads and JALR: the first data memory

2 **Stage 2**

- LB, LH, LW, LBU, LHU with byte and half-word alignment and sign or zero extension
- Read-only data memory, combinational read
- JALR: rd = pc + 4, pc = (rs1 + imm) & ~1, the first instruction that changes the pc
- Still one instruction per cycle

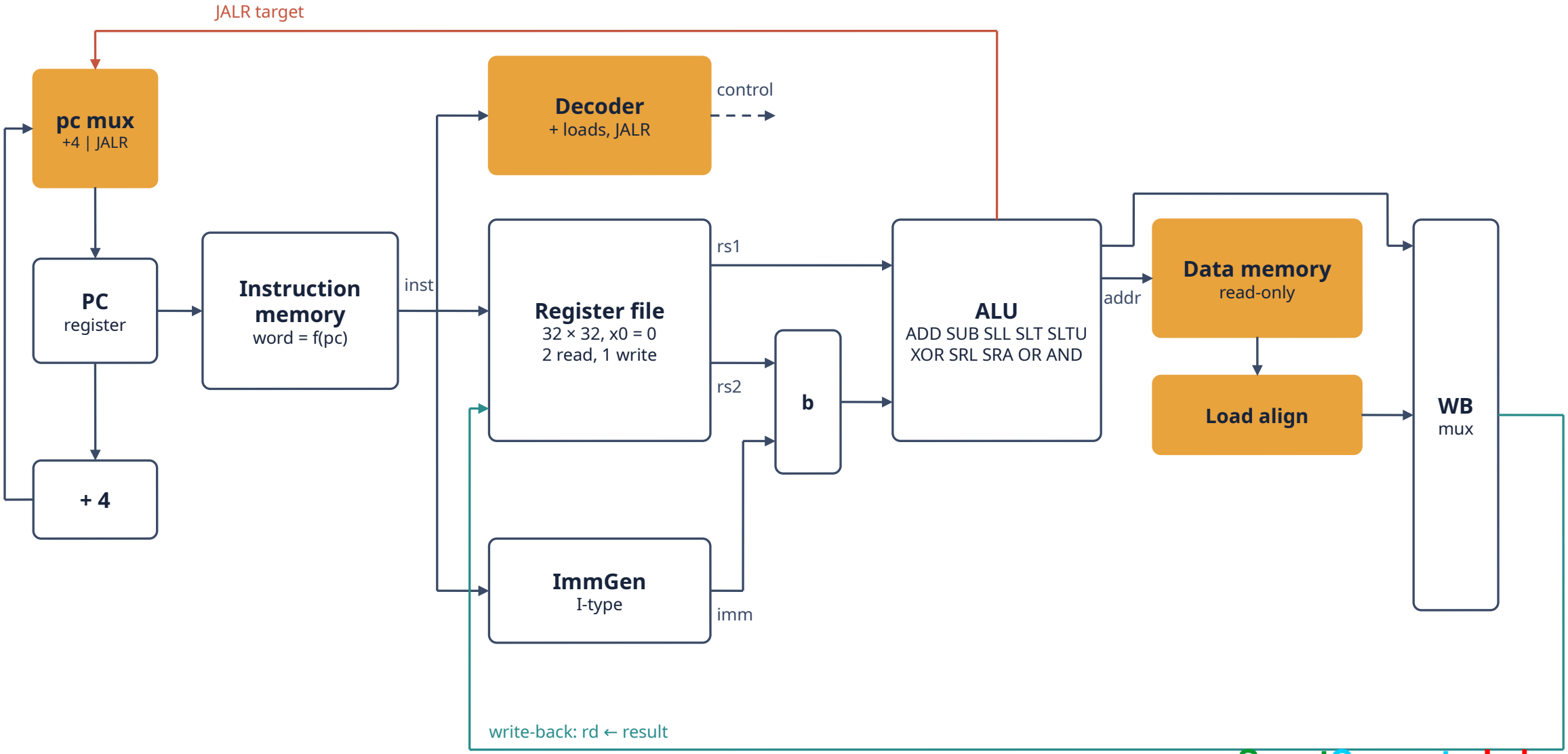


+6 instructions

1 cycle each

33 tests

Stage 2 architecture: a data memory and a pc mux



SmartComputerLab

new in this stage carried over write-back

Complete RV32I + Zicsr with precise traps

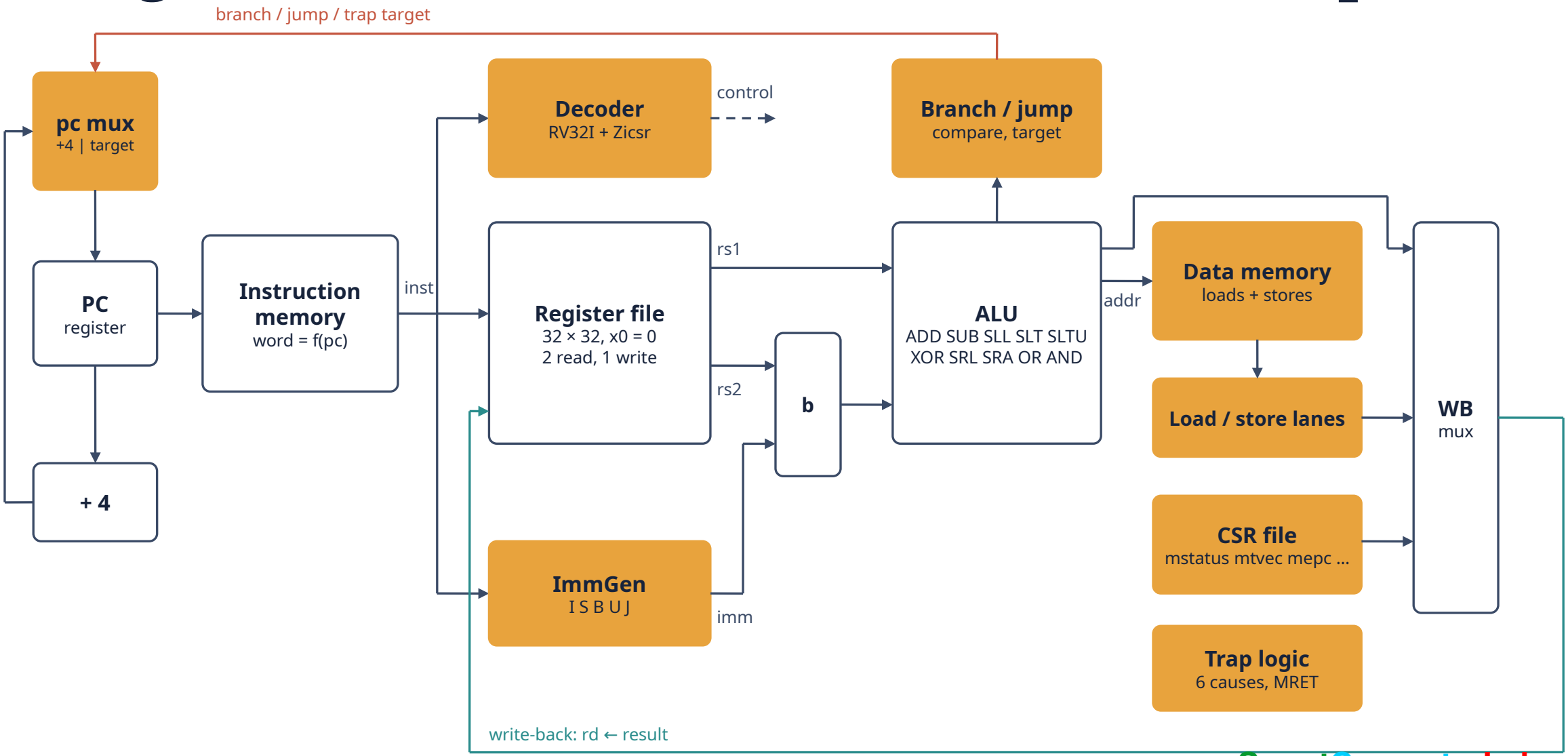
3 **Stage 3**

- Stores SB, SH, SW with byte strobes
- Six branches, JAL, LUI, AUIPC
- Zicsr: CSRRW, CSRRS, CSRRC and the immediate forms; mstatus, mtvec, mepc, mcause, mtval, mcycle, minstret
- A trapping instruction changes nothing; the handler starts at mtvec and returns with MRET

Exceptions (mcause)

0	Instruction address misaligned
2	Illegal instruction
3	Breakpoint (EBREAK)
4	Load address misaligned
6	Store address misaligned
11	ECALL from M-mode

Stage 3 architecture: branches, CSRs and traps



RV32IM multiplication in a single cycle

4 **Stage 4**

- MUL, MULH, MULHSU, MULHU
- One combinational 33 × 33 signed multiplier: each operand extended by one bit, signed or unsigned
- The result is ready in the same cycle as any ALU result
- DIV and REM still trap as illegal instructions

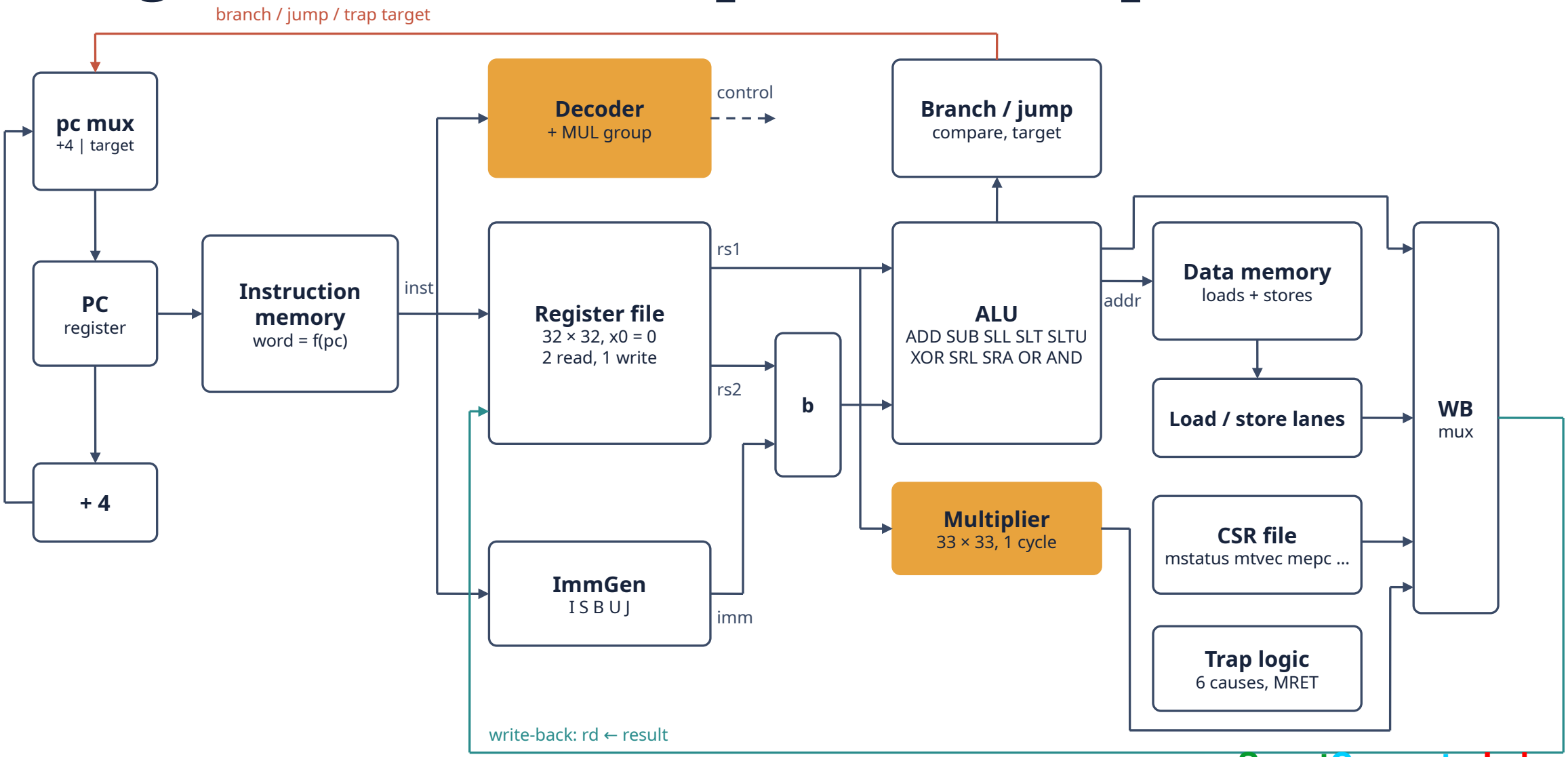
Instruction	rs1	rs2	Result bits
MUL	any	any	31:0
MULH	signed	signed	63:32
MULHSU	signed	unsigned	63:32
MULHU	unsigned	unsigned	63:32

4
new instructions

1
cycle each

34
tests

Stage 4 architecture: a parallel multiplier



A sequential divider completes RV32IM

5 **Stage 5**

- DIV, DIVU, REM, REMU on a divider that produces one quotient bit per cycle
- Division by zero and MIN ÷ −1 give the RISC-V results in 2 cycles
- Only division is sequential: every other instruction still takes one cycle
- misa reads 0x40001100 (RV32IM)

One division on the timeline



The core holds the instruction and retires nothing while it waits.

÷0: quotient −1, remainder = dividend
MIN ÷ −1: quotient MIN, remainder 0

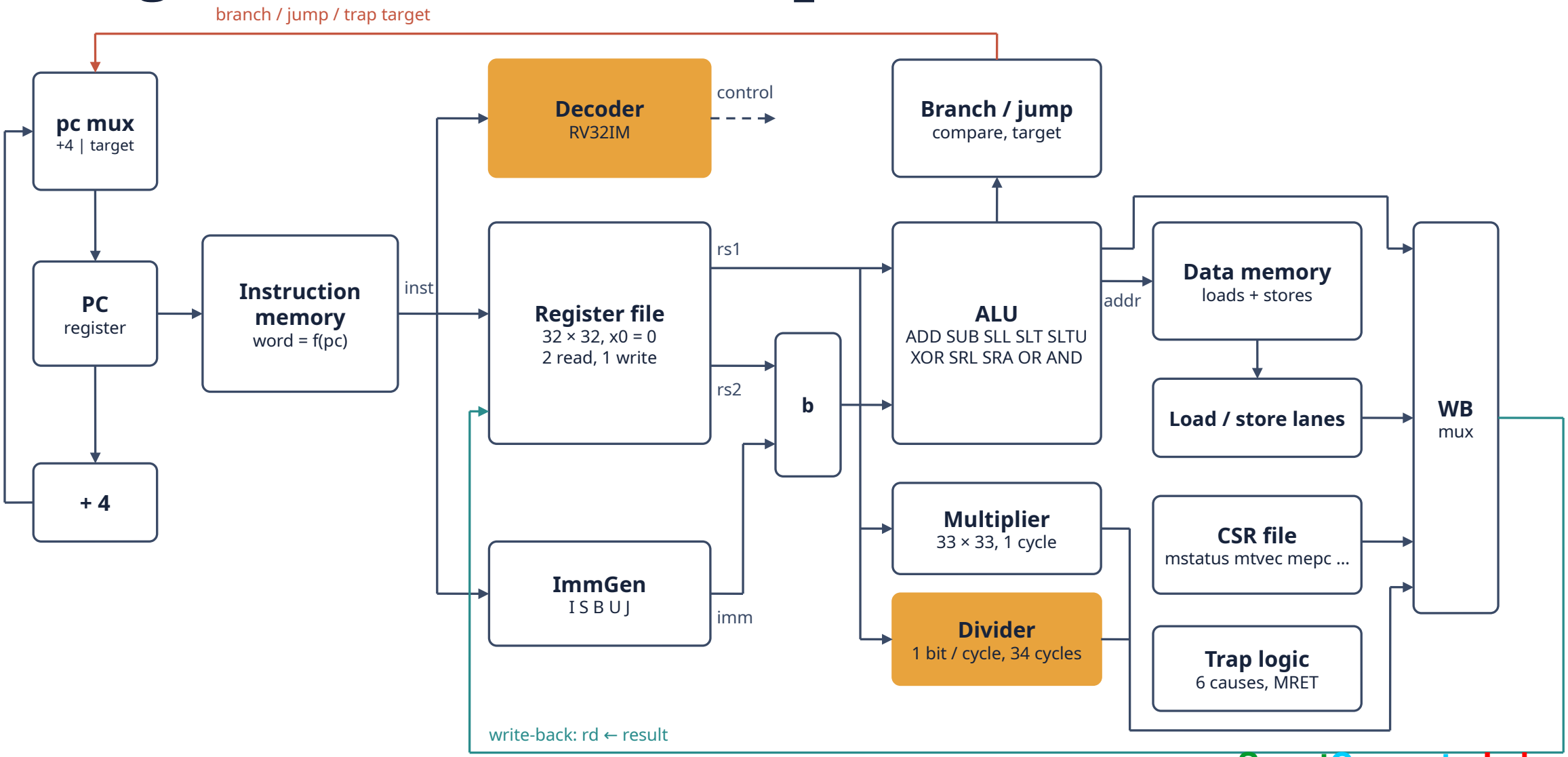
34
cycles per division

2
cycles for ÷0 and overflow

47
tests

SmartComputerLab

Stage 5 architecture: a sequential divider



SmartComputerLab

6–8

Stages 6–8: pipelines and caches

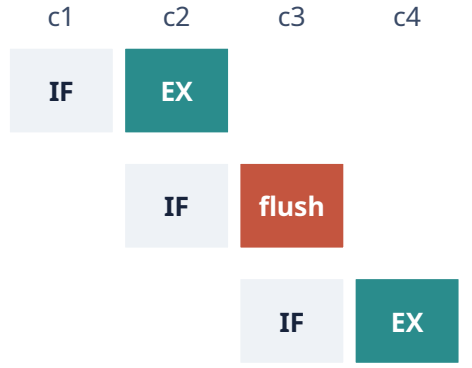
Same RV32IM + Zicsr, new microarchitecture: 2 stages, 5 stages, then caches

Stage 6: a 2-stage pipeline, IF | EX

6 **Stage 6**

- IF: fetchPc addresses the instruction memory; the word lands in IfEx (valid, pc, inst)
- EX: decode, registers, ALU, multiplier, divider, memory, CSR, traps, write-back
- Fetch assumes pc + 4; a taken branch, jump, trap or MRET flushes IfEx: 1 bubble
- During a division IF and EX both hold for 33 cycles

A taken branch costs one bubble

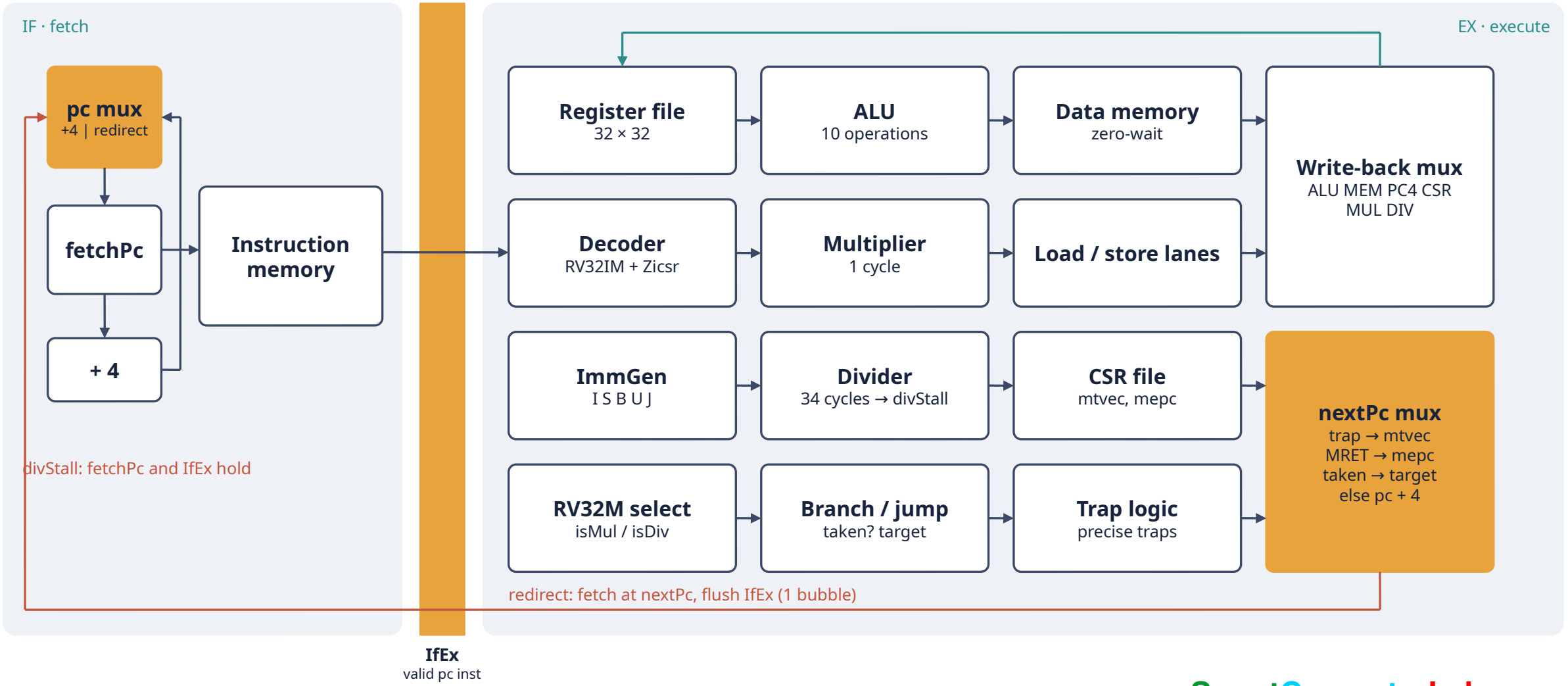


2
stages

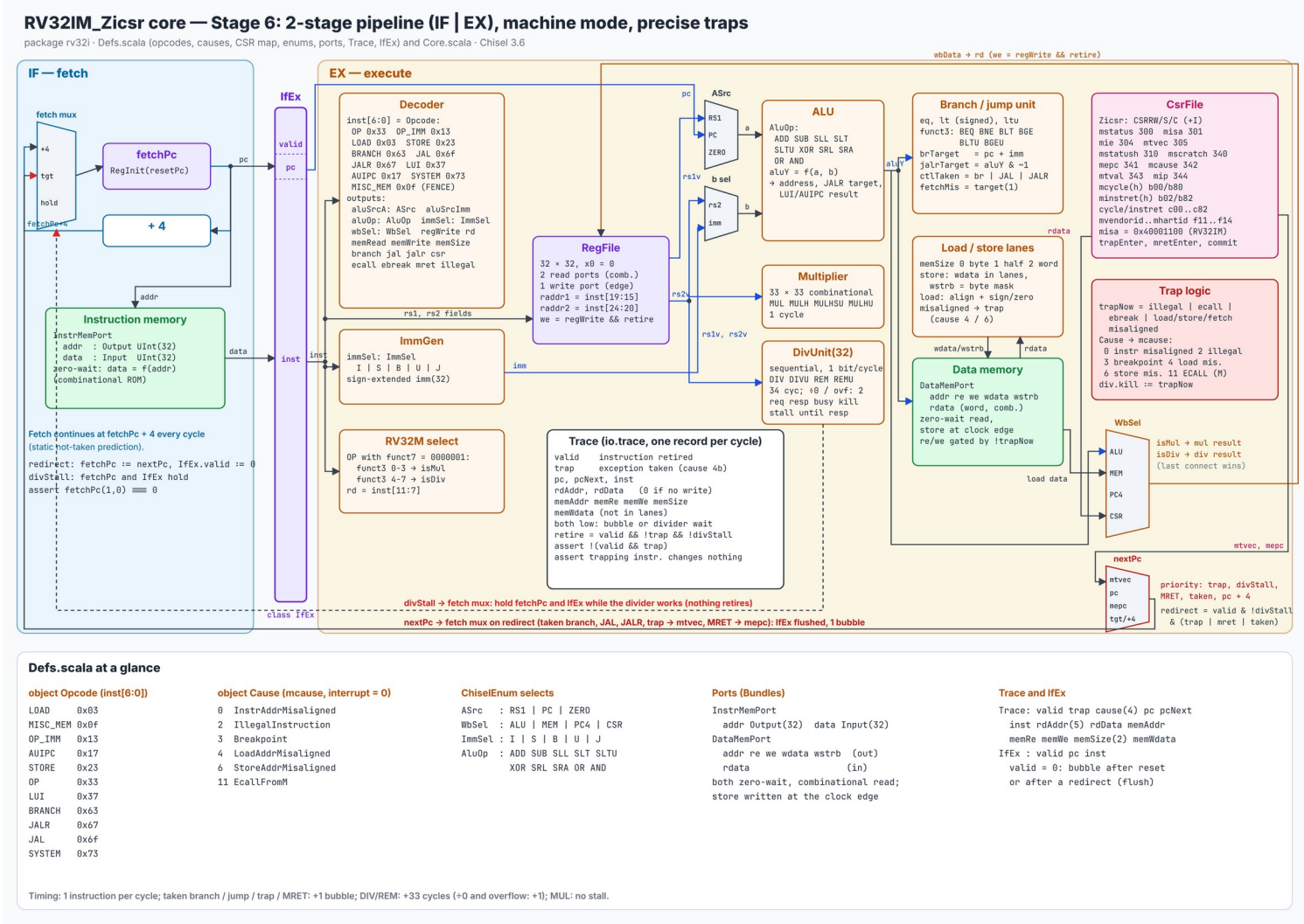
+1
bubble per redirect

48
tests

Stage 6 architecture: IF | IfEx | EX



Stage 6 datapath: Defs.scala and Core.scala in one picture



SmartComputerLab

Stage 7: 5-stage pipeline with forwarding

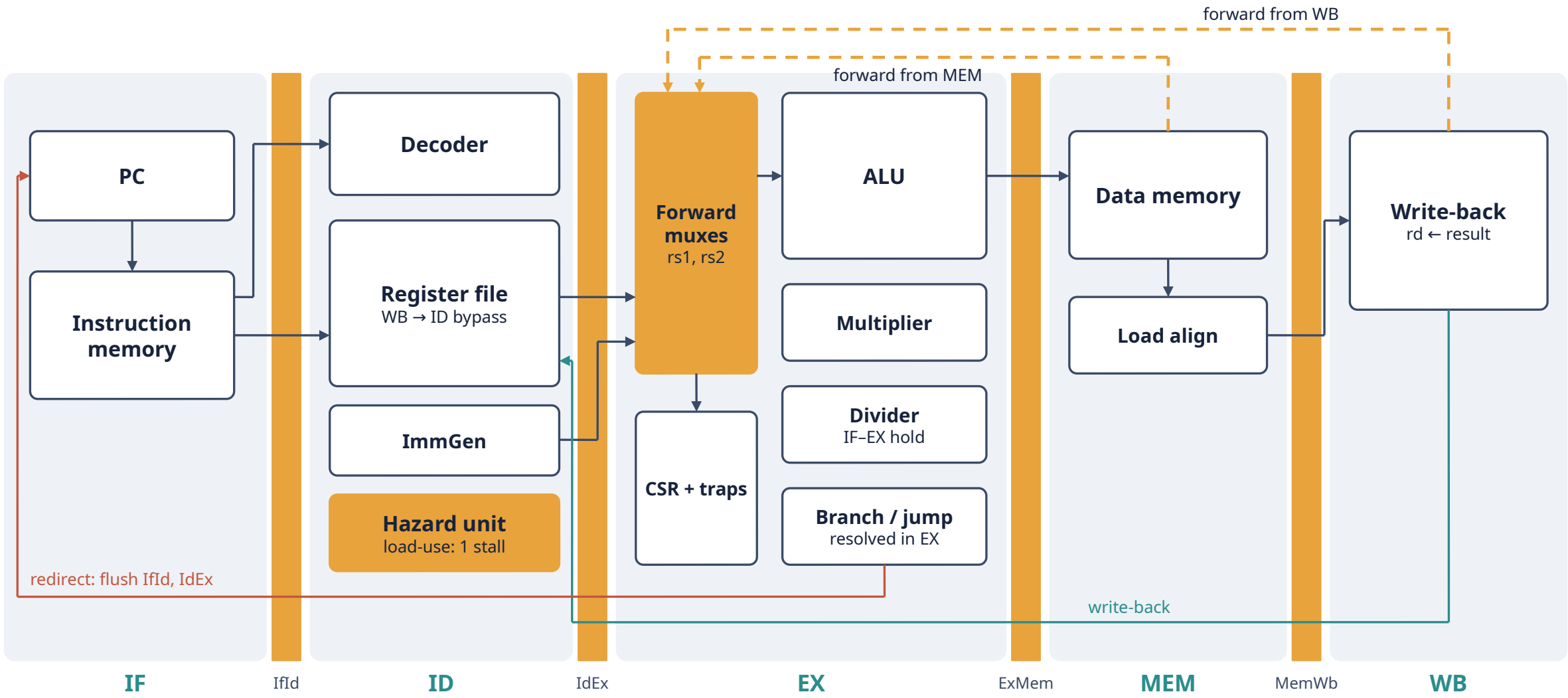
7 **Stage 7**

- IF, ID, EX, MEM, WB with IfId, IdEx, ExMem, MemWb
- Forwarding from MEM and WB, plus a WB → ID bypass
- Branches and jumps resolved in EX; exceptions taken in EX stay precise
- Project: rv32im-pipe5 · 37 tests



Hazard	Handling	Cost
RAW, distance 1	Forward ExMem	0
RAW, distance 2	Forward MemWb	0
RAW, distance 3	WB → ID bypass	0
Load-use	Stall + bubble	1
Taken branch / jump	Flush IfId, IdEx	2
DIV / REM	IF–EX hold	33

Stage 7 architecture: five stages with forwarding



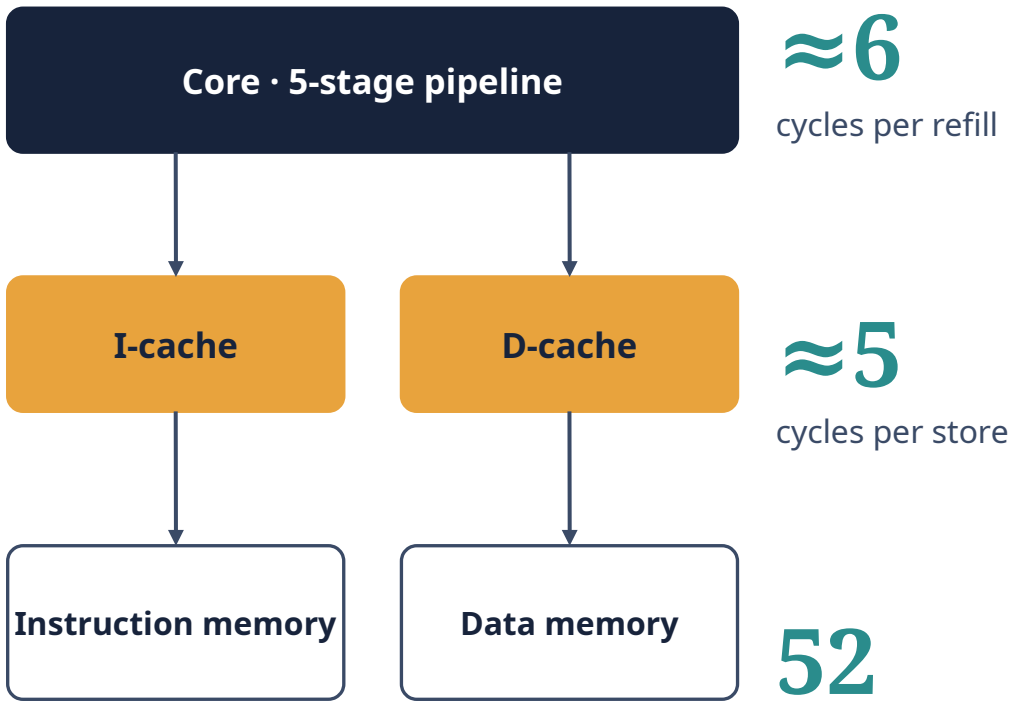
SmartComputerLab

new in this stage · carried over · forwarding

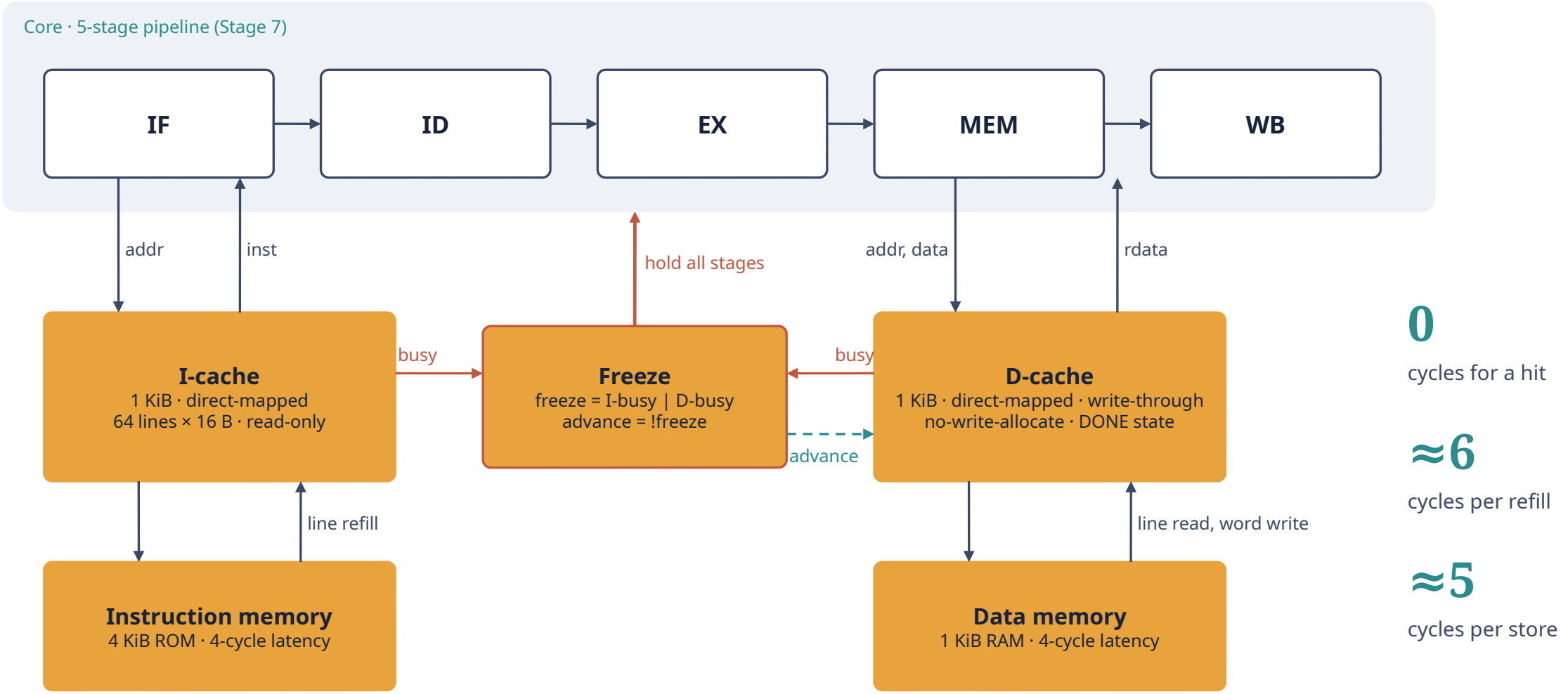
Stage 8: an I-cache and a D-cache

8 **Stage 8**

- Both 1 KiB, direct-mapped, 64 lines × 16 bytes
- D-cache: write-through, no-write-allocate
- Main memories answer after 4 cycles; a hit costs nothing
- While either cache is busy, the whole pipeline freezes



Stage 8 architecture: caches in front of slow memories



new in this stage carried over freeze

SmartComputerLab

A freeze must not repeat a store

Freeze rules

- Every pipeline register holds
- Register write, CSR commit, trap entry and trace are gated
- The divider keeps working and holds its result
- A freeze costs exactly its length when nothing else is in flight

The bug the tests found

1

Cycle 85

D-cache acknowledges the store while the I-cache is still busy: the pipeline stays frozen

2

Cycle 86

The store is still in MEM, so the D-cache sends the same write again

3

Fix

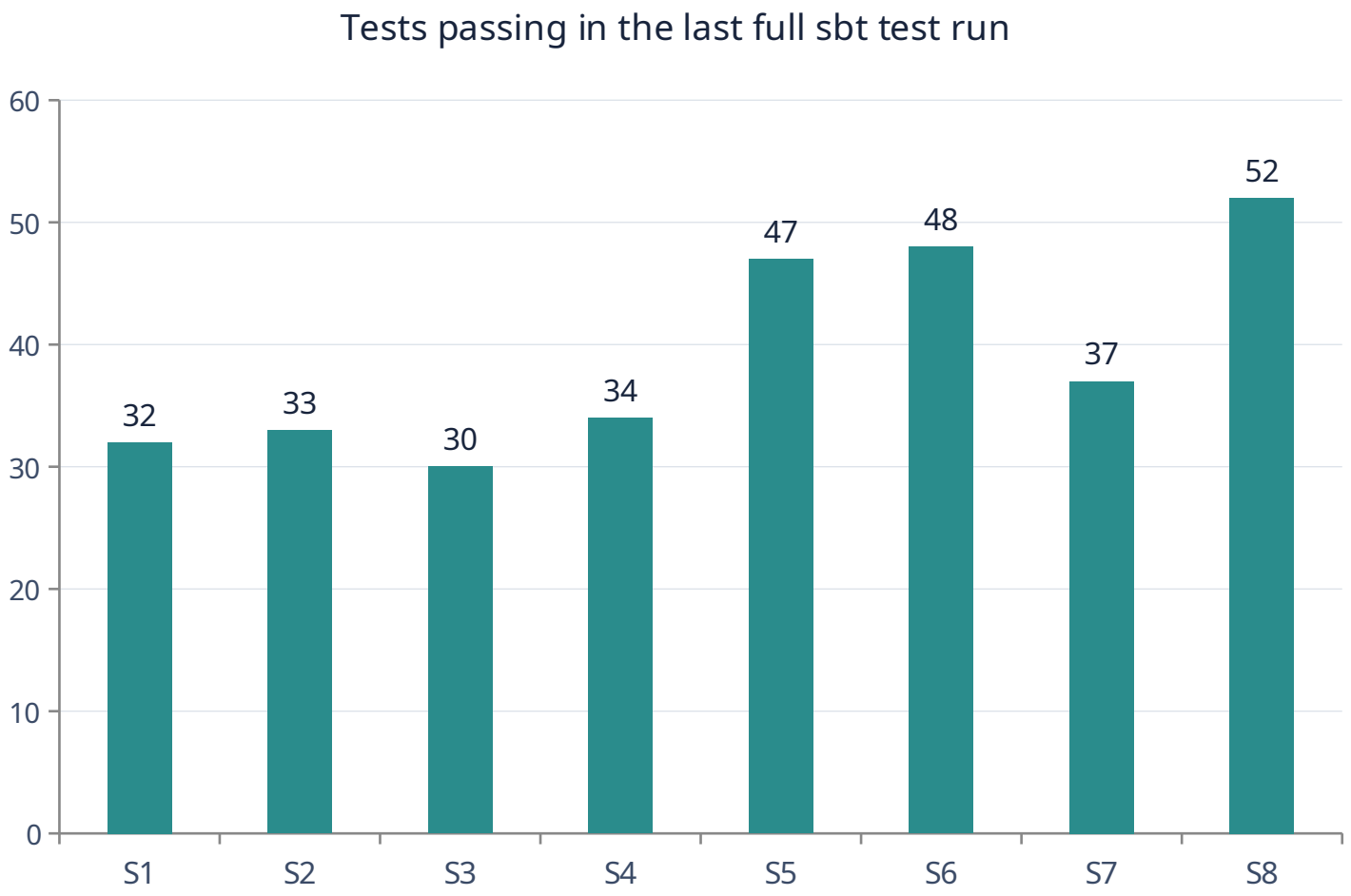
The core drives advance = !freeze; the D-cache waits in DONE until the pipeline moves

The eight stages side by side

Stage	Project	ISA	New hardware	Cycle cost
1	rv32i-stage1	19 ALU instructions	ALU, decoder, immediate, register file	1
2	rv32i-stage2	+ 5 loads, JALR	Read-only data memory, load alignment	1
3	rv32i-stage3	RV32I + Zicsr	Stores, branches, jumps, CSRs, traps	1
4	rv32im-stage4	+ 4 multiplications	Combinational 33 × 33 multiplier	1
5	rv32im-stage5	RV32IM + Zicsr	Sequential divider	1; DIV/REM 34
6	rv32im-stage6	same	2-stage pipeline IF | EX	+1 per redirect
7	rv32im-pipe5	same	5-stage pipeline, forwarding	+1 load-use, +2 redirect
8	rv32im-stage8	same	I-cache, D-cache, freeze	+≈6 per miss, +≈5 per store

SmartComputerLab

Every stage ships with 30–52 automated tests



What the counts include

- Unit tests for every block
- Lockstep programs against RefModel, up to 3,000 random instructions
- Timing tests with exact cycle counts
- Waveform demos checked against RefModel
- Stage 1: 7 suites writing Verilog test vectors
- Stage 8: cache and freeze suites

Four layers of verification in every project

1 Unit tests

Each block against Scala expectations: edge values, thousands of random operands, every divider corner case, each cache against a directory model.

2 Lockstep programs

The core runs next to RefModel; the trace is compared every cycle. Directed programs plus 3,000 random instructions.

3 Timing tests

Exact cycle costs: one per instruction, divider waits, pipeline fill, bubbles, the cost of a cache freeze.

4 Planted bugs

Faults such as SRA decoded as SRL, a cache hit ignoring the tag or a store written twice must each fail the suite.

Waveforms to look at, Verilog to take away

GTKWave sessions

- Demo programs write VCD files to waves/
- core.gtkw groups signals by pipeline stage
- Registers in decimal, instructions shown as assembly through translate filters
- Stage 8 adds I-cache and D-cache groups: state, hit, refill, counters

Verilog generation

- Emit: SystemVerilog through firtool, or Chisel 3's built-in compiler
- EmitSplit (Stage 1): one .v or .sv file per module
- Seven self-checking Verilog testbenches, one per module plus core and system
- make, or make SIM=verilator

From 19 instructions to a cached pipeline, never leaving a known-good state

Next: branch prediction · a write buffer · a set-associative or write-back D-cache · interrupts and a timer · FENCE.I